Tecnológico de Monterrey, Monterrey Campus

Applications of Cryptography and Cybersecurity

DNS Servers with BIND: Authoritative and Recursive Configuration, Root Hints, and Traffic Analysis

---

Abstract — The goal of this activity is to study how DNS servers are configured with BIND, to distinguish authoritative from recursive servers, to understand the role of `root.hints`, and to analyze the structure of DNS responses with Wireshark and `dig`. The BIND and `root.hints` sections are theoretical, while the traffic analysis was carried out in practice on a real domain. This work is the foundation for a later lab in which DNSSEC will be implemented.

Keywords: DNS, BIND, Wireshark, dig, DNSSEC

Team 1

Elías Alejandro Corral Rodríguez	A01743624
Jannette Cristina Flores Rodríguez	A01255711
Angel Eduardo Luna	A00842657
Montserrat Rodríguez Ramírez	A01661271
Carolina Sarahí Sánchez Meneses	A00227980

Professors: Mohd Anas Wajid

Date: November 12, 2026

Contents

1. Introduction 3
   1.1. Objectives 3
   1.2. Environment used for the traffic analysis, ... 3
2. What Is BIND and How Is It Used in DNS? 3
   2.1. Overview 3
   2.2. Main functions 4
3. BIND Configuration Options 4
   3.1. Configuration files involved 4
   3.2. Authoritative server 5
   3.3. Recursive server 6
   3.4. Authoritative vs. recursive comparison 7
   3.5. Configuration verification 7
4. Importance of the `root.hints` File 7
   4.1. Role in iterative resolution 7
5. DNS Query Analysis with Wireshark and `dig` 8
   5.1. Methodology 8
   5.2. Sections of a DNS response message 8
   5.3. SOA records 11
   5.4. NS records 12
   5.5. A records 15
   5.6. AAAA records 16
   5.7. TTL summary by record 18
6. Results and Discussion 18
   6.1. Authoritative vs. recursive servers 18
   6.2. Effect of `root.hints` 19
   6.3. Analysis of the DNS response 19
   6.4. TTL behavior 20
7. Conclusions 20
8. References 21
9. Declaration of AI Use 21

1. Introduction

The Domain Name System (DNS) is a distributed, hierarchical system that maps domain names to IP addresses (Mockapetris, 1987a). Thanks to it, users only need to remember a name (for example, `tec.mx`) instead of a numeric address, much like contacts on a phone replaced memorizing phone numbers. Its predecessor was the `hosts.txt` file, a central list of name-to-address mappings that stopped being viable as the network grew, because it did not scale and was hard to keep up to date (Mockapetris, 1987a).

When a user types a name in the browser, a recursive resolver queries the DNS hierarchy (the root server, the top-level domain (TLD) server, and the domain's authoritative server) and returns the IP address. The answer is stored in a cache for a period called the TTL (*Time to Live*), so later queries are answered faster and with less load on the servers (Mockapetris, 1987a).

The goal of this activity is to learn how DNS is configured in a test lab using BIND, to distinguish authoritative from recursive servers, to understand the role of `root.hints`, and to analyze the structure of DNS responses with Wireshark and `dig`. This work is the foundation for a later lab in which DNSSEC will be implemented.

1.1. Objectives

- Describe how BIND is configured as an *authoritative* and as a *recursive* server, and which configuration files are involved.
- Explain the role of the `root.hints` file in recursive resolution.
- Analyze DNS responses with Wireshark and `dig` (message sections and SOA, NS, A, and AAAA records, including their TTL).

1.2. Environment used for the traffic analysis,

Component	**Details**
Operating system	MacOS 15.6.1 (24G90)
`dig` version	dig 9.10.6
Wireshark version	Version 4.6.9 (v4.6.9-0-g2d548b197c75).
Topology / IP addresses	Client: MacOS laptop (10.23.76.254) connected through Wi-Fi (`en0`). Recursive resolver: the network's default resolver (10.23.56.43, port 53). Authoritative servers of `typst.app`: `ns.inwx.de.` (192.174.68.104), `ns2.inwx.de.` and `ns3.inwx.eu.`, reached by the resolver on the client's behalf.

Table 1: Summary of the environment used for the traffic analysis.

2. What Is BIND and How Is It Used in DNS?

2.1. Overview

BIND (*Berkeley Internet Name Domain*) is the most widely used DNS server software on the Internet. It originated at the University of California, Berkeley in the 1980s and is now developed

and maintained by the Internet Systems Consortium (ISC). The current major version is BIND 9, and its main service runs as the `named` process (Internet Systems Consortium, n.d.).

BIND implements the standard DNS protocols and is used both to publish information about a network's own domains and to resolve names on behalf of its clients. It also ships with supporting tools: `dig` (queries), `rndc` (service control), `named-checkconf` and `named-checkzone` (configuration and zone validation), and utilities for DNSSEC (Internet Systems Consortium, n.d.).

2.2. Main functions

Function	**Description**
Authoritative server	Stores the records of the zones it manages and answers with authority for them (responses carry the `aa` flag).
Recursive server (resolver)	Resolves client queries by walking the DNS hierarchy (root, TLD, authoritative) and returns the final answer.
Cache	Stores answers for their TTL so repeated queries are answered faster.
Others (forwarding, views, DNSSEC, zone transfers)	*Forwarding*: pass queries on to another resolver. Views: answer differently depending on the client. DNSSEC: sign and validate responses. Zone transfers: copy zones between a primary and secondary servers.

Table 2: BIND functions in a DNS environment.

3. BIND Configuration Options

3.1. Configuration files involved

On Debian, Ubuntu, and Kali the BIND configuration is split into several files inside `/etc/bind/`; on RHEL-type systems everything is usually kept in `/etc/named.conf`. The table shows which file controls each part of the server (Internet Systems Consortium, n.d.).

File	Typical location	Purpose
`named.conf`	`/etc/bind/named.conf` (RHEL: `/etc/named.conf`)	Main file. On Debian-style systems it only includes the other files; on RHEL-type systems it holds the whole configuration.
`named.conf.options`	`/etc/bind/named.conf.options`	Global options (`directory`, `recursion`, `allow-query`, `forwarders`, `dnssec-validation`). The **recursive** mode is configured here.
`named.conf.local`	`/etc/bind/named.conf.local`	Declaration of the zones the server owns (primary/secondary). The **authoritative** mode is configured here.
Zone file (`db.example.com`)	`/etc/bind/db.example.com` (RHEL: `/var/named/`)	The records of the zone: SOA, NS, A, AAAA, etc.
`root.hints`	`/usr/share/dns/root.hints` (or the file named in the `"."` zone)	List of root servers used to start a resolution. BIND has a built-in copy if the file is not declared (Internet Systems Consortium, 2009).

Table 3: Relevant BIND configuration files.

3.2. Authoritative server

An authoritative server holds the original data for one or more zones. To configure it, each zone is declared in `named.conf.local` with `type master` (or `primary` in recent versions) and the path of its zone file. The `allow-transfer` option restricts who may copy the zone. A server that is only authoritative should disable recursion with the option `recursion no`, because it should not resolve names for third parties, and leaving recursion on exposes it to abuse (open resolver) (Internet Systems Consortium, n.d.).

```
// named.conf.local  (example)
zone "example.com" {
    type master;
    file "/etc/bind/db.example.com";
    allow-transfer { none; };
};
```

```
// named.conf.options  (authoritative-ONLY server)
options {
    directory "/var/cache/bind";
    recursion no;
    allow-query { any; };
};
```

```
; db.example.com - zone file (example)
$TTL 3600
@   IN  SOA ns1.example.com. admin.example.com. (
            2026110601 ; serial
            3600       ; refresh
            900        ; retry
            604800     ; expire
            300 )      ; negative TTL
@       IN  NS    ns1.example.com.
ns1     IN  A     192.168.1.10
www     IN  A     192.168.1.20
www     IN  AAAA  2001:db8::20
```

The fields of the SOA record are:

Field	**Meaning**
Primary server	`ns1.example.com.`, the main name server of the zone.
Responsible email	`admin.example.com.` (the first dot stands for the at-sign).
Serial	Version number of the zone; it must increase with every change.
Refresh	How often the secondary servers check for changes.
Retry	How often they retry if the check fails.
Expire	When they stop answering if they cannot reach the primary.
Negative TTL	How long the fact that a name does not exist is cached (Andrews, 1998).

Table 4: Fields of the SOA record.

3.3. Recursive server

A recursive server receives client queries and resolves them on their behalf. It is configured in `named.conf.options` with the option `recursion yes`. The options `allow-recursion` and `allow-query-cache` limit who may use it, so only the clients of its own network can. Optionally, `forwarders` hand resolution over to another server, and `dnssec-validation` enables DNSSEC signature validation (Internet Systems Consortium, n.d.). This type of server needs a list of root servers to know where to start: BIND's built-in list, or a `root.hints` file declared explicitly (Internet Systems Consortium, 2009).

```
// named.conf.options  (recursive server)
options {
    directory "/var/cache/bind";
    recursion yes;
    allow-recursion { 192.168.1.0/24; localhost; };
    listen-on { any; };
    // forwarders { 8.8.8.8; };   // optional
    dnssec-validation auto;
};
```

`allow-recursion` is restricted because a recursive resolver open to the whole Internet can be abused by attackers for amplification (DDoS) attacks and as a target for cache poisoning attempts.

3.4. Authoritative vs. recursive comparison

Aspect	Authoritative	Recursive
`recursion`	`no`	`yes`
Own zones	Yes, declared with `zone` and a zone file	Not necessarily
Needs `root.hints`	No	Not strictly (a built-in copy is used if none is declared)
Typical response flag	`aa` (authoritative answer)	`ra` (recursion available)
Files to modify	`named.conf.local` and the zone file (plus `named.conf.options` to set `recursion no`)	`named.conf.options`

Table 5: Configuration differences between the two modes.

3.5. Configuration verification

```
named-checkconf                              # validates named.conf
named-checkzone example.com /etc/bind/db.example.com
sudo systemctl restart named                 # or bind9, depending on the distro
sudo systemctl status named
```

4. Importance of the `root.hints` File

A recursive resolver starts with an empty cache and does not know any DNS server, so it needs a list of root servers to know where to begin any resolution. BIND includes a built-in list, and `root.hints` is the way to declare that list explicitly. In BIND it is declared as a root zone of type `hint` (Internet Systems Consortium, n.d.):

```
zone "." {
    type hint;
    file "/usr/share/dns/root.hints";
};
```

4.1. Role in iterative resolution

When a recursive resolver has no cached answer, it resolves a name step by step, and `root.hints` provides the first step:

1. The resolver asks a root server (taken from `root.hints`), which answers with the servers of the TLD (for example `.mx`).
2. The resolver asks the TLD server, which answers with the authoritative servers of the domain.

3. The resolver asks the authoritative server, which gives the final answer.
4. The resolver returns the answer to the client and stores it in its cache for the TTL.

BIND ships with a built-in copy, so `root.hints` is not strictly required, but declaring it explicitly allows the list to be updated or replaced, which is essential in an isolated network (for example, a private test lab) where the real root servers are unreachable and the file must point to that network's own root servers. If the declared file is missing, empty, or lists unreachable servers, the resolver may not be able to start the resolution chain and BIND reports errors. Root server addresses change very rarely, but the file should still be kept up to date. An up-to-date copy is published by IANA (IANA, n.d.) and can also be obtained with:

```
# Obtain an up-to-date copy of the file
dig +bufsize=1200 +norec NS . @a.root-servers.net > root.hints
```

5. DNS Query Analysis with Wireshark and `dig`

5.1. Methodology

The analysis was carried out against a real domain, `typst.app`, through the network's recursive resolver, to observe the structure of real DNS traffic.

- Domain queried: `typst.app`
- DNS server queried: the network's recursive resolver (10.23.56.43), which is the default resolver of the client; `dig` was run without specifying a server (`@server`).
- Client: macOS laptop (10.23.76.254) with `dig` 9.10.6.
- Wireshark capture interface: Wi-Fi (`en0`). Display filter: `dns.qry.name == "typst.app"`.

```
dig typst.app SOA
dig typst.app NS
dig typst.app A
dig typst.app AAAA
```

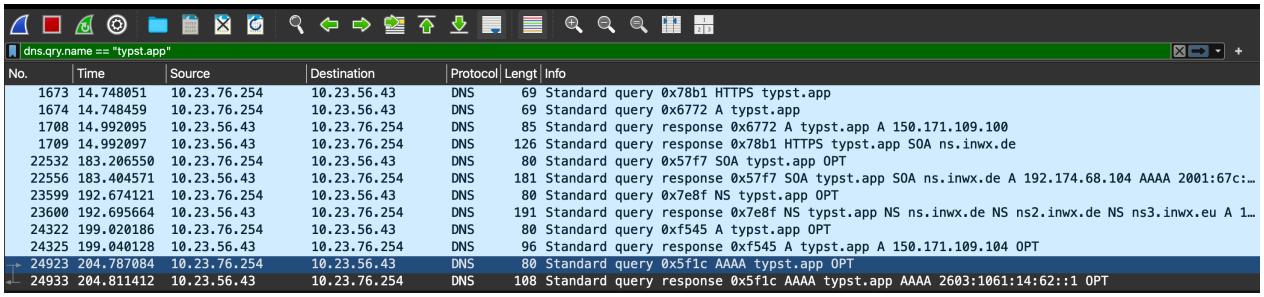


Figure 1: DNS packets for `typst.app` captured on Wi-Fi (`en0`) with the filter `dns.qry.name == "typst.app"`. The last eight packets correspond to the four `dig` queries and their responses.

5.2. Sections of a DNS response message

A DNS response message has four sections (Mockapetris, 1987b).

Section	Description	Observed in the capture
Question	Repeats the original DNS query made, confirms the parameters.	`typst.app. IN NS`: name `typst.app`, type NS, class IN (`Questions: 1`).
Answer	Contains the DNS resource records that match the lookup request.	Three NS records (`Answer RRs: 3`), all with TTL 4076 s (1 h 7 min 56 s): `ns.inwx.de.`, `ns2.inwx.de.`, `ns3.inwx.eu.`
Authority	Lists the name servers that are authoritative for the zone or to which the query is delegated; empty when the answer is complete.	Empty (`Authority RRs: 0`).
Additional	Auxiliary resource records that were not explicitly requested but help the client avoid extra queries.	Three records (`Additional RRs: 3`): an A record (`ns.inwx.de.` → 192.174.68.104), an AAAA record (`ns.inwx.de.` → 2001:67c:1bc::104), and the EDNS OPT pseudo-record. The two address records have TTL 291 s.

Table 6: Sections of the DNS response message.

The response header (flags `0x8180`) has the QR, RD, and RA bits set and the AA bit clear (Mockapetris, 1987b). This means the answer came from a recursive resolver (10.23.56.43) and not directly from an authoritative server of `typst.app`, which is consistent with the `ra` flag discussed in Section 3. The TTL of 4076 s is most likely the remaining time of a cached copy, not the value configured in the zone. The Authority section is empty because the response already contains the requested records and no referral to another server is needed, while the Additional section carries the addresses of `ns.inwx.de` so the client can contact it without a new query.

```
> User Datagram Protocol, Src Port: 53, Dst Port: 60057
v Domain Name System (response)
    Transaction ID: 0x7e8f
  v Flags: 0x8180 Standard query response, No error
        1... .... .... .... = Response: Message is a response
        .000 0... .... .... = Opcode: Standard query (0)
        .... .0.. .... .... = Authoritative: Server is not an authority for domain
        .... ..0. .... .... = Truncated: Message is not truncated
        .... ...1 .... .... = Recursion desired: Do query recursively
        .... .... 1... .... = Recursion available: Server can do recursive queries
        .... .... .0.. .... = Z: reserved (0)
        .... .... ..0. .... = Answer authenticated: Answer/authority portion was not authenticated by the server
        .... .... ...0 .... = Non-authenticated data: Unacceptable
        .... .... .... 0000 = Reply code: No error (0)
    Questions: 1
    Answer RRs: 3
    Authority RRs: 0
    Additional RRs: 3
  v Queries
    > typst.app: type NS, class IN
  v Answers
    v typst.app: type NS, class IN, ns ns.inwx.de
        Name: typst.app
        Type: NS (2) (authoritative Name Server)
        Class: IN (0x0001)
        Time to live: 4076 (1 hour, 7 minutes, 56 seconds)
        Data length: 12
        Name Server: ns.inwx.de
    v typst.app: type NS, class IN, ns ns2.inwx.de
        Name: typst.app
        Type: NS (2) (authoritative Name Server)
        Class: IN (0x0001)
        Time to live: 4076 (1 hour, 7 minutes, 56 seconds)
        Data length: 6
        Name Server: ns2.inwx.de
    v typst.app: type NS, class IN, ns ns3.inwx.eu
        Name: typst.app
        Type: NS (2) (authoritative Name Server)
        Class: IN (0x0001)
        Time to live: 4076 (1 hour, 7 minutes, 56 seconds)
        Data length: 13
        Name Server: ns3.inwx.eu
  v Additional records
    v ns.inwx.de: type A, class IN, addr 192.174.68.104
        Name: ns.inwx.de
        Type: A (1) (Host Address)
        Class: IN (0x0001)
        Time to live: 291 (4 minutes, 51 seconds)
        Data length: 4
        Address: 192.174.68.104
    v ns.inwx.de: type AAAA, class IN, addr 2001:67c:1bc::104
        Name: ns.inwx.de
        Type: AAAA (28) (IP6 Address)
        Class: IN (0x0001)
        Time to live: 291 (4 minutes, 51 seconds)
        Data length: 16
        AAAA Address: 2001:67c:1bc::104
    v <Root>: type OPT
        Name: <Root>
        Type: OPT (41)
        UDP payload size: 4000
        Higher bits in extended RCODE: 0x00
        EDNS0 version: 0
      > Z: 0x0000
        Data length: 0
    [Request In: 23599]
    [Time: 21.543000 milliseconds]
```

Figure 2: DNS response packet with its four sections.

5.3. SOA records

The SOA (*Start of Authority*) record marks the start of a zone and holds its administrative parameters; each zone has exactly one (Mockapetris, 1987b). The query `dig typst.app SOA` returned one SOA record in the Answer section (`ANSWER: 1`), whose fields are summarized below.

Field	Value	Meaning
Primary server (MNAME)	`ns.inwx.de.`	Main name server of the zone.
Responsible party (RNAME)	`hostmaster.inwx.de.`	Contact email, read as `hostmaster@inwx.de` (the first dot stands for the at-sign).
Serial	`2026101700`	Zone version. It follows the `YYYYMMDDnn` convention: October 17, 2026, revision 00.
Refresh	10800 s (3 h)	How often secondary servers check the primary for changes.
Retry	3600 s (1 h)	How often they retry if the check fails.
Expire	604800 s (7 days)	When secondary servers stop answering if they cannot reach the primary.
Minimum / negative TTL	3600 s (1 h)	How long the non-existence of a name may be cached (Andrews, 1998).

Table 7: Fields of the SOA record of `typst.app`.

The record itself has a TTL of 86400 s (24 hours), which is different from the *minimum* field (3600 s): the first controls how long the SOA record can be cached, while the second applies to negative answers. The Additional section of this response carried the A and AAAA records of the primary server `ns.inwx.de.` (TTL 300 s), so the client can reach it without another query. The response was slower (198 ms) than the other queries (about 20 ms), which suggests that the resolver had to fetch it from the authoritative server instead of its cache.

```
user@host ~ % dig typst.app SOA

; <<>> DiG 9.10.6 <<>> typst.app SOA
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 22519
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 3

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4000
;; QUESTION SECTION:
;typst.app.                     IN      SOA

;; ANSWER SECTION:
typst.app.              86400   IN      SOA     ns.inwx.de. hostmaster.inwx.de. 2026101700 10800 3600 604800 3600

;; ADDITIONAL SECTION:
ns.inwx.de.             300     IN      A       192.174.68.104
ns.inwx.de.             300     IN      AAAA    2001:67c:1bc::104

;; Query time: 198 msec
;; SERVER: 10.23.56.43#53(10.23.56.43)
;; WHEN: Sun Nov 08 18:22:08 CST 2026
;; MSG SIZE  rcvd: 139
```

Figure 3: SOA record query and response in Terminal.

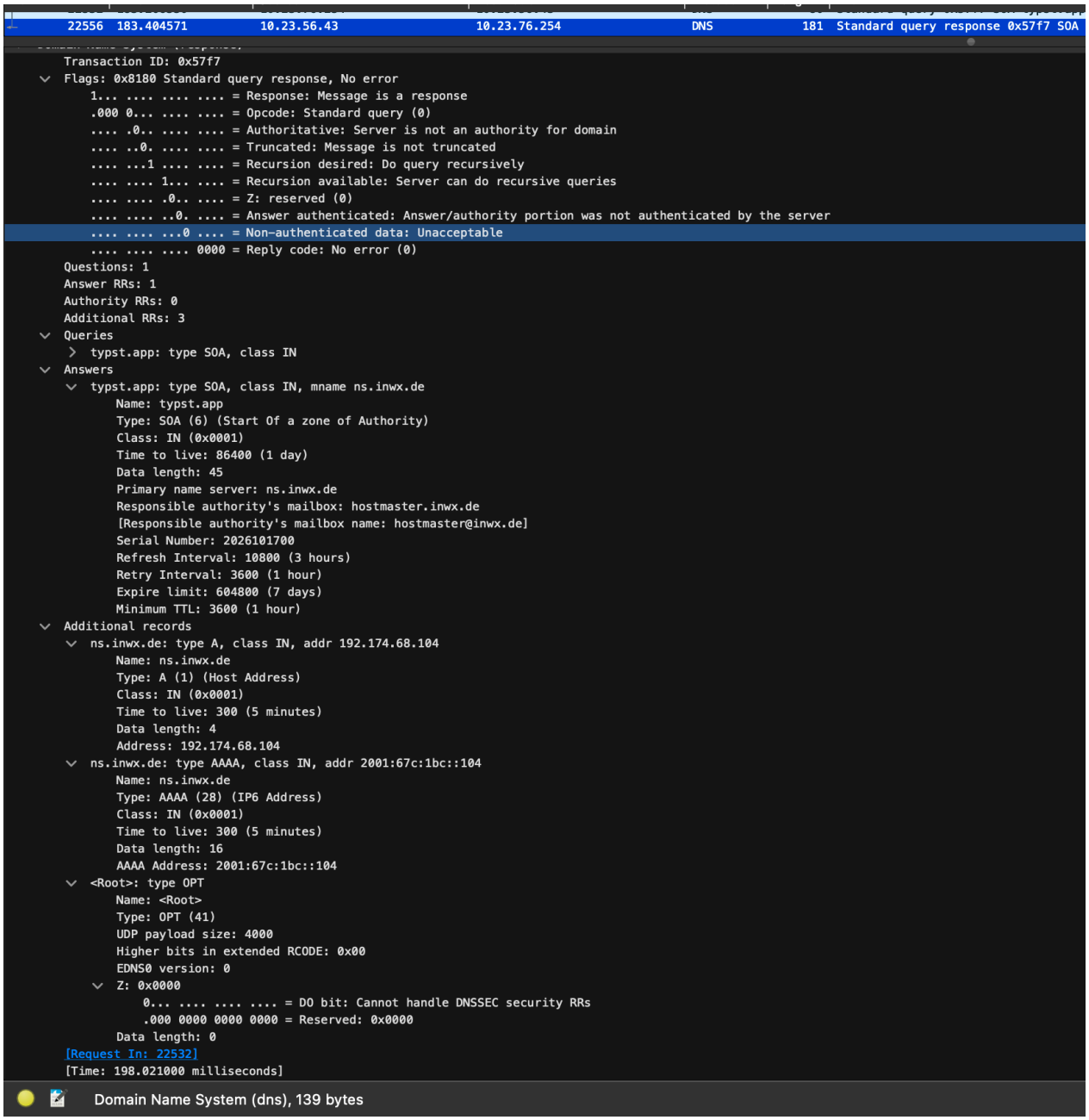


Figure 4: SOA record query and response in Wireshark.

5.4. NS records

An NS record names a server that is authoritative for a domain, and it is the mechanism of DNS delegation: the NS records of a zone, published in the parent zone and in the zone itself, tell resolvers which servers to ask for that zone's data (Mockapetris, 1987a). The query for `typst.app` returned three NS records (`Answer RRs: 3`), `ns.inwx.de.`, `ns2.inwx.de.`, and `ns3.inwx.eu.`, all with a TTL of 4076 s (1 hour, 7 minutes, 56 seconds). Having several name servers, in different domains (`.de` and `.eu`), provides redundancy: if one fails, the zone can still be resolved through the others.

The response was marked with the `rd` and `ra` flags and without `aa`, so it was delivered by the network's recursive resolver (10.23.56.43) and not directly by an authoritative server. The Authority section is empty (`Authority RRs: 0`) because the answer is already complete. The Additional section carried the A (192.174.68.104) and AAAA (2001:67c:1bc::104) records of `ns.inwx.de.`, both with TTL 291 s, plus the EDNS OPT pseudo-record. The TTL of 4076 s is not a round number, which indicates that it is the remaining time of a cached copy.

```
user@host ~ % dig typst.app NS
[
; <<>> DiG 9.10.6 <<>> typst.app NS
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 32399
;; flags: qr rd ra; QUERY: 1, ANSWER: 3, AUTHORITY: 0, ADDITIONAL: 3

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4000
;; QUESTION SECTION:
;typst.app.                     IN      NS

;; ANSWER SECTION:
typst.app.              4076    IN      NS      ns.inwx.de.
typst.app.              4076    IN      NS      ns2.inwx.de.
typst.app.              4076    IN      NS      ns3.inwx.eu.

;; ADDITIONAL SECTION:
ns.inwx.de.             291     IN      A       192.174.68.104
ns.inwx.de.             291     IN      AAAA    2001:67c:1bc::104

;; Query time: 21 msec
;; SERVER: 10.23.56.43#53(10.23.56.43)
;; WHEN: Sun Nov 08 18:22:17 CST 2026
;; MSG SIZE  rcvd: 149
```

Figure 5: NS record query and response in Terminal.

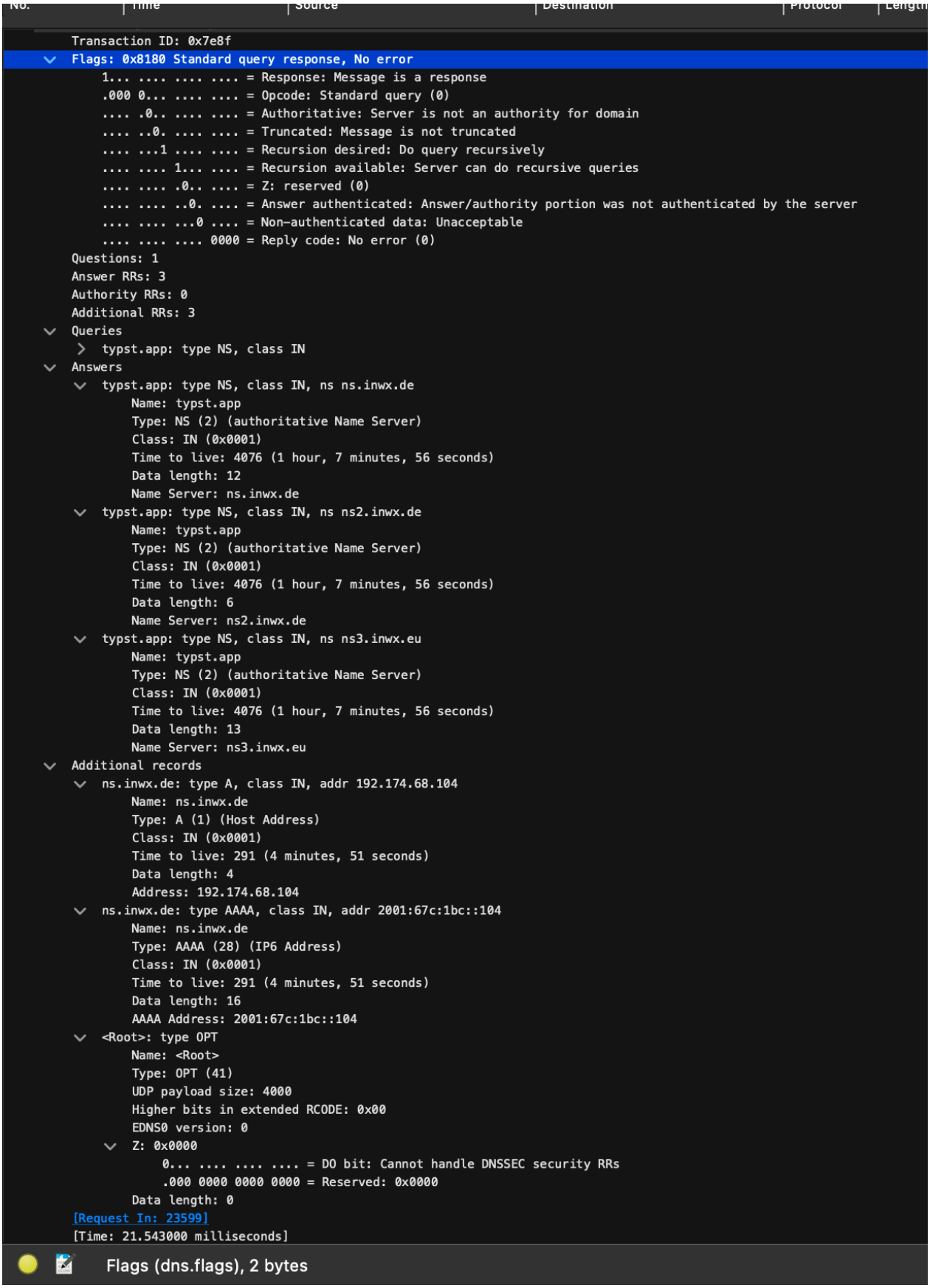


Figure 6: NS record query and response in Wireshark.

5.5. A records

An A record maps a name to an IPv4 address (32 bits) (Mockapetris, 1987b). The query `dig typst.app A` returned a single A record (`Answer RRs: 1`): `typst.app` resolves to `150.171.109.104`, with a TTL of 300 s (5 minutes). The response has `Authority RRs: 0`, and its only Additional record is the EDNS OPT pseudo-record. In that record, the DO bit is 0, which means the client did not request DNSSEC data, something that will change in the next lab. As in the previous queries, the `aa` flag is not set and `ra` is set, so the answer came from the recursive resolver. The query took about 20 ms.

```
user@host ~ % dig typst.app A
[
; <<>> DiG 9.10.6 <<>> typst.app A
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 62789
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4000
;; QUESTION SECTION:
;typst.app.                     IN      A

;; ANSWER SECTION:
typst.app.              300     IN      A       150.171.109.104

;; Query time: 20 msec
;; SERVER: 10.23.56.43#53(10.23.56.43)
;; WHEN: Sun Nov 08 18:22:23 CST 2026
;; MSG SIZE  rcvd: 54
```

Figure 7: A record query and response in Terminal.

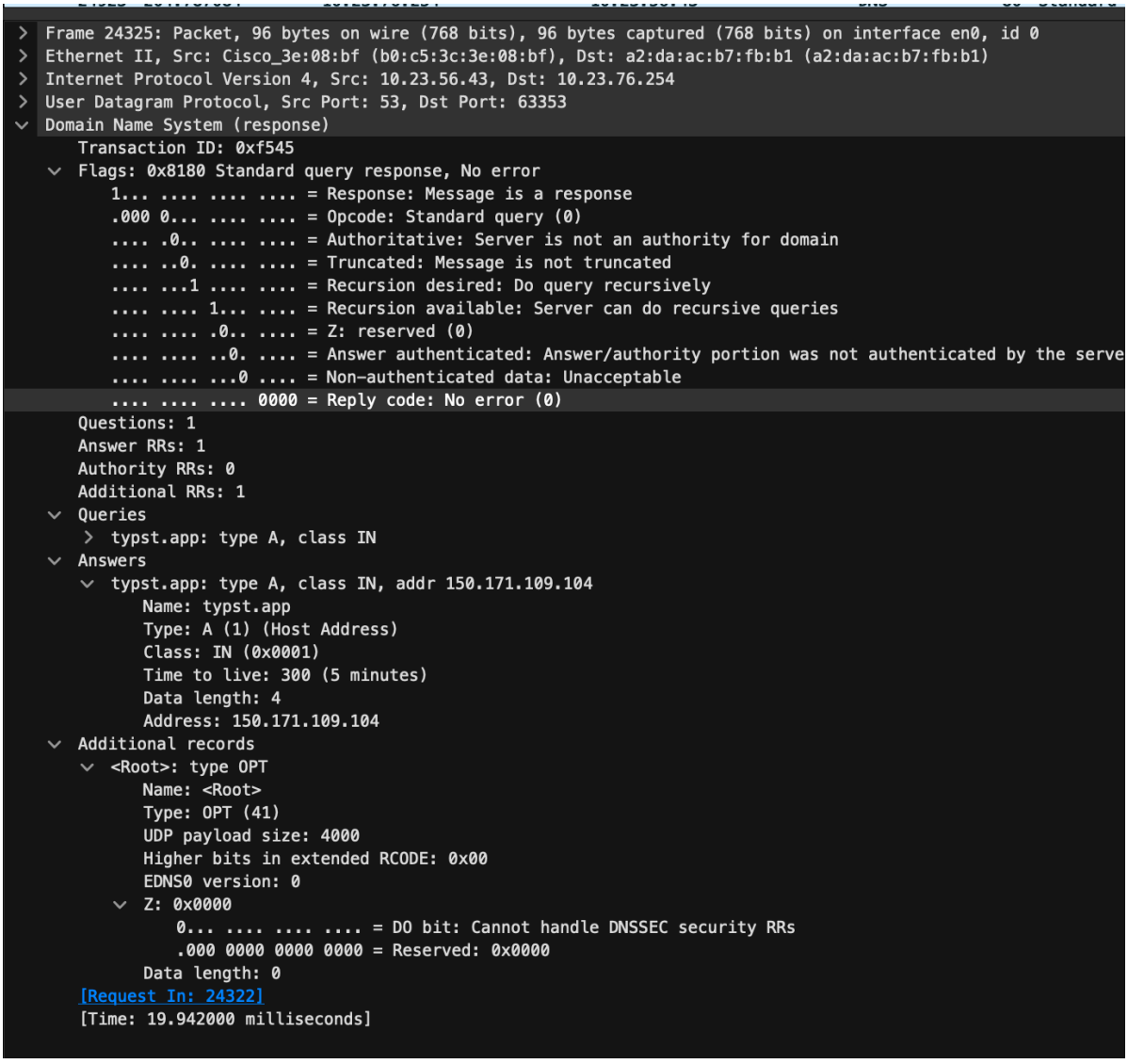


Figure 8: A record query and response in Wireshark.

5.6. AAAA records

An AAAA record maps a name to an IPv6 address (128 bits), and it plays the same role for IPv6 that the A record plays for IPv4. The query `dig typst.app AAAA` returned a single record (`Answer RRs: 1`): `typst.app` resolves to `2603:1061:14:62::1`, with a TTL of 300 s (5 minutes), the same as its A record. The structure of the response is the same as for the A query: no Authority records, the OPT pseudo-record as the only Additional record, and the `ra` flag set. The fact that the domain has both A and AAAA records means that it can be reached from IPv4 and IPv6 networks (dual stack).

```
user@host ~ % dig typst.app AAAA

; <<>> DiG 9.10.6 <<>> typst.app AAAA
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 24348
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4000
;; QUESTION SECTION:
;typst.app.                     IN      AAAA

;; ANSWER SECTION:
typst.app.              300     IN      AAAA    2603:1061:14:62::1

;; Query time: 24 msec
;; SERVER: 10.23.56.43#53(10.23.56.43)
;; WHEN: Sun Nov 08 18:22:29 CST 2026
;; MSG SIZE  rcvd: 66
```

Figure 9: AAAA record query and response in Terminal.

```
Domain Name System (response)
    Transaction ID: 0x5f1c
    Flags: 0x8180 Standard query response, No error
        1... .... .... .... = Response: Message is a response
        .000 0... .... .... = Opcode: Standard query (0)
        .... .0.. .... .... = Authoritative: Server is not an authority for domain
        .... ..0. .... .... = Truncated: Message is not truncated
        .... ...1 .... .... = Recursion desired: Do query recursively
        .... .... 1... .... = Recursion available: Server can do recursive queries
        .... .... .0.. .... = Z: reserved (0)
        .... .... ..0. .... = Answer authenticated: Answer/authority portion was not authenticated by the server
        .... .... ...0 .... = Non-authenticated data: Unacceptable
        .... .... .... 0000 = Reply code: No error (0)
    Questions: 1
    Answer RRs: 1
    Authority RRs: 0
    Additional RRs: 1
    Queries
        typst.app: type AAAA, class IN
    Answers
        typst.app: type AAAA, class IN, addr 2603:1061:14:62::1
            Name: typst.app
            Type: AAAA (28) (IP6 Address)
            Class: IN (0x0001)
            Time to live: 300 (5 minutes)
            Data length: 16
            AAAA Address: 2603:1061:14:62::1
    Additional records
        <Root>: type OPT
            Name: <Root>
            Type: OPT (41)
            UDP payload size: 4000
            Higher bits in extended RCODE: 0x00
            EDNS0 version: 0
            Z: 0x0000
                0... .... .... .... = DO bit: Cannot handle DNSSEC security RRs
                .000 0000 0000 0000 = Reserved: 0x0000
            Data length: 0
    [Request In: 24923]
    [Time: 24.328000 milliseconds]
```

Figure 10: AAAA record query and response in Wireshark.

5.7. TTL summary by record

The TTL (*Time to Live*) indicates how many seconds a record may be cached before it must be queried again (Mockapetris, 1987a).

Type	Name	TTL (s)	Value / Notes
SOA	`typst.app.`	86400	`ns.inwx.de.` `hostmaster.inwx.de.`, serial 2026101700, minimum 3600 s
NS	`typst.app.`	4076	`ns.inwx.de.`, `ns2.inwx.de.`, `ns3.inwx.eu.`
A	`typst.app.`	300	`150.171.109.104`
AAAA	`typst.app.`	300	`2603:1061:14:62::1`
A (additional)	`ns.inwx.de.`	300 / 291	`192.174.68.104`. 300 s in the SOA response, 291 s in the NS response
AAAA (additional)	`ns.inwx.de.`	300 / 291	`2001:67c:1bc::104`. Same behavior as the A record above

Table 8: Observed TTL for each record type in the domain `typst.app`.

The TTLs fall into two groups. The SOA (86400 s), A (300 s), and AAAA (300 s) records have round values, which is consistent with the TTLs configured in the zone. In contrast, the NS records show 4076 s, which is not a round number: it is the remaining time of a copy that was already cached and is counting down.

The captures also show the countdown directly. The A and AAAA records of `ns.inwx.de.` appeared with a TTL of 300 s in the SOA response (frame 22556, `dig` run at 18:22:08) and with 291 s in the NS response (frame 23600, `dig` run at 18:22:17). The difference of 9 s matches the time elapsed between both queries (about 9.3 s according to the Wireshark timestamps), which confirms that the resolver was serving a cached copy whose TTL decreased with time. When it reaches zero, the resolver must query the authoritative server again.

The record TTL must not be confused with the *minimum* field of the SOA record. The first controls how long an existing record can be cached; for example, the SOA record itself has a TTL of 86400 s (24 hours). The second, 3600 s in this zone, is used for negative caching: according to RFC 2308 (Andrews, 1998), the resolver caches the non-existence of a name for the lower of the SOA record's TTL and its minimum field, which here is 3600 s (1 hour).

6. Results and Discussion

6.1. Authoritative vs. recursive servers

The study of BIND's configuration shows that the two server roles are separated both by purpose and by the files that define them. An authoritative server is defined in `named.conf.local` (zone declaration) and in the zone file, and it answers only for the zones it owns, which is why its responses carry the `aa` flag; if it is authoritative-only, it also sets `recursion no;` in `named.conf.options`. A recursive server is defined in `named.conf.options` with the option `recursion yes`, and it answers

on behalf of its clients by walking the hierarchy and caching the results, so its answers carry the `ra` flag.

The captures of Section 5 show the recursive side. Every response had the `rd` and `ra` flags set and `aa` clear, so it came from the network's recursive resolver (10.23.56.43) and not from an authoritative server of `typst.app`. An `aa` flag would only appear in an answer received directly from one of the domain's name servers, which was not captured in this activity.

A server that is only authoritative should use `recursion no`. A recursive server should limit its clients with `allow-recursion`; otherwise it becomes an open resolver that can be abused for amplification attacks and cache poisoning attempts.

6.2. Effect of `root.hints`

The `root.hints` file provides a recursive resolver, which starts from an empty cache, with the addresses of the root servers (through this file or BIND's built-in copy). The root servers return the referral to the TLD servers, which in turn return the referral to the authoritative servers of the domain. An authoritative-only server does not need it, since it never resolves names for third parties. In an isolated network the file takes on a critical role: the built-in list of BIND points to the real root servers, which would be unreachable, so `root.hints` must point to that network's own root servers. If the file is missing, empty, or lists unreachable servers, the resolution chain may not start and `named` reports errors.

6.3. Analysis of the DNS response

The query for the NS records of `typst.app` returned a response with the four sections of a DNS message, summarized below.

Section	Count	Content observed
Question	1	`typst.app. IN NS`
Answer	3	NS records `ns.inwx.de.`, `ns2.inwx.de.`, and `ns3.inwx.eu.`, all with TTL 4076 s
Authority	0	Empty
Additional	3	A (192.174.68.104) and AAAA (2001:67c:1bc::104) records of `ns.inwx.de.` (TTL 291 s) and the EDNS OPT pseudo-record

Table 9: Summary of the response sections observed in the capture.

The Answer section contains exactly what was asked for, while the Additional section carries data that saves the client an extra query: the IPv4 and IPv6 addresses of `ns.inwx.de.` come attached so that the client can contact that name server right away. The Authority section is empty because the response already contains the requested records and no delegation to another server is needed. The header flags (`qr rd ra`, without `aa`) show that the response came from the network's recursive resolver (10.23.56.43) and not directly from an authoritative server of `typst.app`.

6.4. TTL behavior

The records observed have different TTLs (4076 s for the NS records, 300 s for A and AAAA, and 86400 s for the SOA), which confirms that the TTL is a property of each record set and not of the whole message. The NS value is most likely a remaining time, not the configured one, because it is not a round number and was probably served from the resolver's cache. The captures show the countdown directly: the addresses of `ns.inwx.de.` went from 300 s to 291 s in the 9 seconds between two queries. Once the TTL reaches zero, the resolver must query the authoritative server again. This is the mechanism by which caching reduces the load on the servers and speeds up repeated queries; it also means that a change in a zone can take up to the old TTL to reach every resolver.

The record TTL must not be confused with the *minimum* field of the SOA record. The first controls how long a record that exists may be cached, while the second (3600 s in `typst.app`) is used for negative caching: according to RFC 2308 (Andrews, 1998), a resolver caches the non-existence of a name for the lower of the SOA record's TTL and its minimum field. Here that is the lower of 86400 s and 3600 s, so a negative answer would be cached for 3600 s (1 hour).

7. Conclusions

1. BIND is a flexible DNS server in which the same `named` process can act as an authoritative server, a recursive resolver, or both. The role is defined by a few clearly separated parts of the configuration: the zone declarations (`named.conf.local`) and zone files for the authoritative role, and the `recursion` and `allow-recursion` options (`named.conf.options`) for the recursive role.
2. Separating both roles is a security best practice. An authoritative-only server should use `recursion no`, and a recursive server should restrict its clients with `allow-recursion`, so that neither becomes an open resolver that can be abused for amplification attacks or cache poisoning attempts.
3. The `root.hints` file is the starting point of any recursive resolution. Although BIND includes a built-in copy, declaring it explicitly allows the list to be updated or replaced, which is essential in an isolated network where the real root servers are unreachable.
4. The captures of `typst.app` made with `dig` and Wireshark showed the four sections of a DNS response (Question, Answer, Authority, and Additional). They also showed how the Additional section carries the address records of `ns.inwx.de.` and the EDNS OPT pseudo-record, which saves the client extra queries.
5. The TTL is a property of each record set: 86400 s for the SOA, 4076 s for the NS records, and 300 s for the A and AAAA records. The addresses of `ns.inwx.de.` went from 300 s to 291 s in the 9 seconds between two queries, which showed the cache countdown directly and explains why a change in a zone can take up to the old TTL to reach every resolver.
6. The record TTL and the SOA *minimum* field (3600 s) control different things: the first applies to records that exist, and the second, according to RFC 2308 (Andrews, 1998), applies to negative caching, that is, to how long the non-existence of a name is remembered.
7. The OPT record showed the DO bit set to 0, meaning that no DNSSEC data was requested. The knowledge gained in this activity (zone files, recursion, root hints, DNS message structure, and TTL behavior) is the foundation for the next lab, in which DNSSEC will be implemented.

8. References

Andrews, M. (1998). *Negative Caching of DNS Queries (DNS NCACHE)* (Request for Comments No. 2308). https://doi.org/10.17487/RFC2308

Anthropic. (2026,). *Claude Sonnet 5.5*. https://claude.ai/

IANA. (n.d.). *Root Files (\texttt{named.root})*. Retrieved November 6, 2026, from https://www.iana.org/domains/root/files

Internet Systems Consortium. (n.d.). *BIND 9 Administrator Reference Manual*. Retrieved November 6, 2026, from https://bind9.readthedocs.io/

Internet Systems Consortium. (2009, August). *BIND 9.x and Hint File*. https://lists.isc.org/pipermail/bind-users/2009-August/077611.html

Mockapetris, P. (1987a). *Domain Names — Concepts and Facilities* (Request for Comments No. 1034). https://doi.org/10.17487/RFC1034

Mockapetris, P. (1987b). *Domain Names — Implementation and Specification* (Request for Comments No. 1035). https://doi.org/10.17487/RFC1035

9. Declaration of AI Use

We used Claude Sonnet 5.5 (Anthropic, 2026) between November 6 and 9, 2026. We used it to:

- **Understand concepts** from class (protocols, ports, DNS records, TTL, DNSSEC).
- **Draft the theory sections** of the report in Typst, which the team then edited and corrected.
- **Review our draft** for technical errors, inconsistencies, and claims our captures did not support.
- **Format** tables, code blocks, and references.

We did not use it to produce data: the `dig` queries and Wireshark captures were done by team members, and all values in Section 5 come from them. We did not accept its output by default; for example, we checked the `root.hints` claim against an ISC source (Internet Systems Consortium, 2009) and corrected an early draft. The team is responsible for the content of this report.